



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Fransiska Agustina Bunjamin
NPM	2428240154
Kelas	SI5C
Topik Tugas 1	Laundry
Entitas utama	Laundry
Tautan repositori	https://github.com/fransiskaab/si5c_kuis1_Fransiska
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NPM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau <i>curl</i> di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap

Bab	Isi	Bukti utama
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Laundry
Entitas utama (tunggal)	Laundry
Alamat dasar	/laundry
Field data	id, namaPelanggan, jenisLayanan, berat, totalHarga
Field wajib saat POST	namaPelanggan, jenisLayanan, berat, totalHarga
Query string untuk filter	Tidak ada
Data awal	id 1 Fransiska Cuci Kering 3 kg Rp21.000; id 2 Cika Cuci Setrika 4 kg Rp32.000; id 3 Andi Cuci Kering 2 kg Rp14.000
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/laundry	Tidak	200	Ambil semua data laundry
2	GET	/laundry/:id	Tidak	200	Ambil satu data laundry berdasarkan ID
3	POST	/laundry	Ya	201	Menambah data laundry
4	PUT	/laundry/:id	Ya	200	Mengubah data laundry
5	DELETE	/laundry/:id	Ya	204	Menghapus data laundry tanpa response body

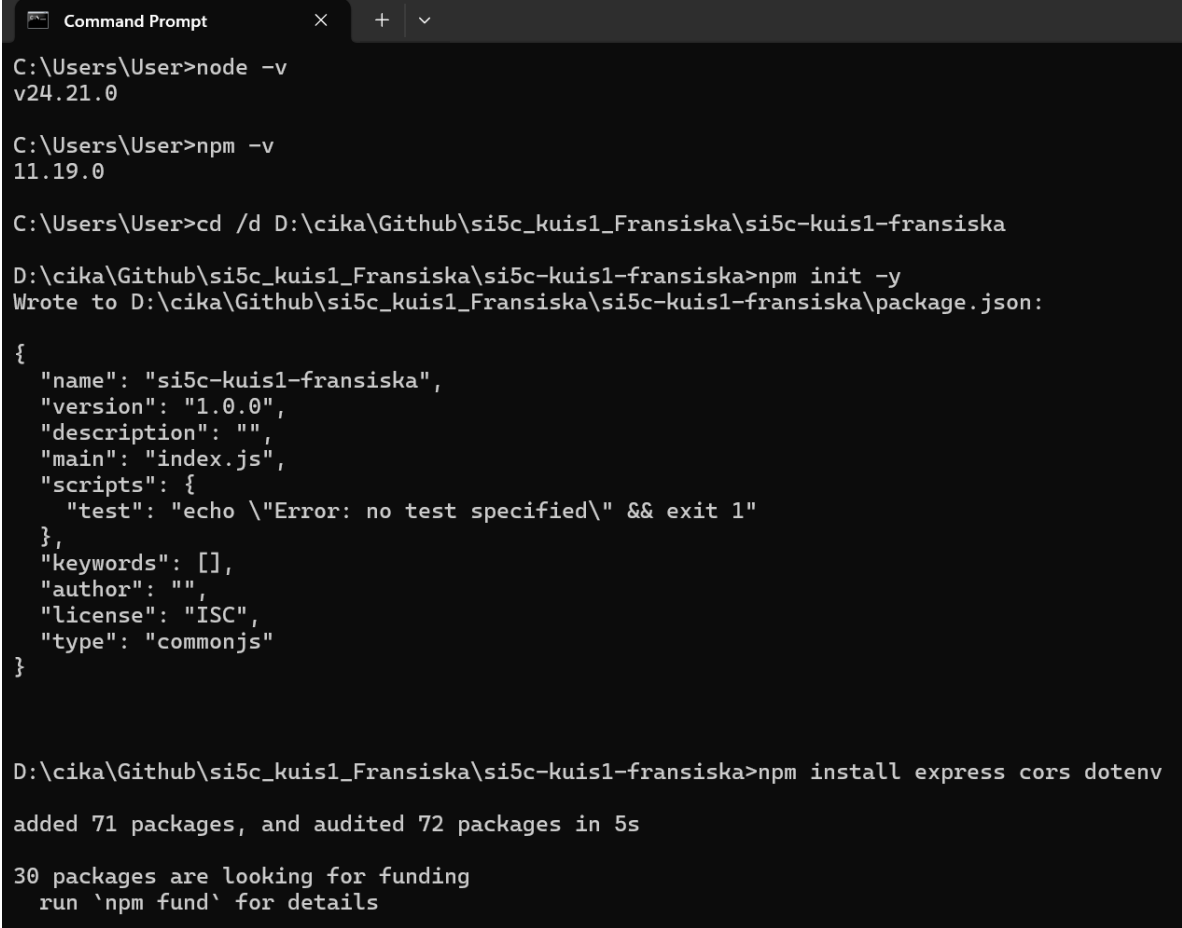
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.



```
Command Prompt
C:\Users\User>node -v
v24.21.0

C:\Users\User>npm -v
11.19.0

C:\Users\User>cd /d D:\cika\Github\si5c_kuis1_Fransiska\si5c-kuis1-fransiska

D:\cika\Github\si5c_kuis1_Fransiska\si5c-kuis1-fransiska>npm init -y
Wrote to D:\cika\Github\si5c_kuis1_Fransiska\si5c-kuis1-fransiska\package.json:

{
  "name": "si5c-kuis1-fransiska",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

D:\cika\Github\si5c_kuis1_Fransiska\si5c-kuis1-fransiska>npm install express cors dotenv

added 71 packages, and audited 72 packages in 5s

30 packages are looking for funding
  run `npm fund` for details
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

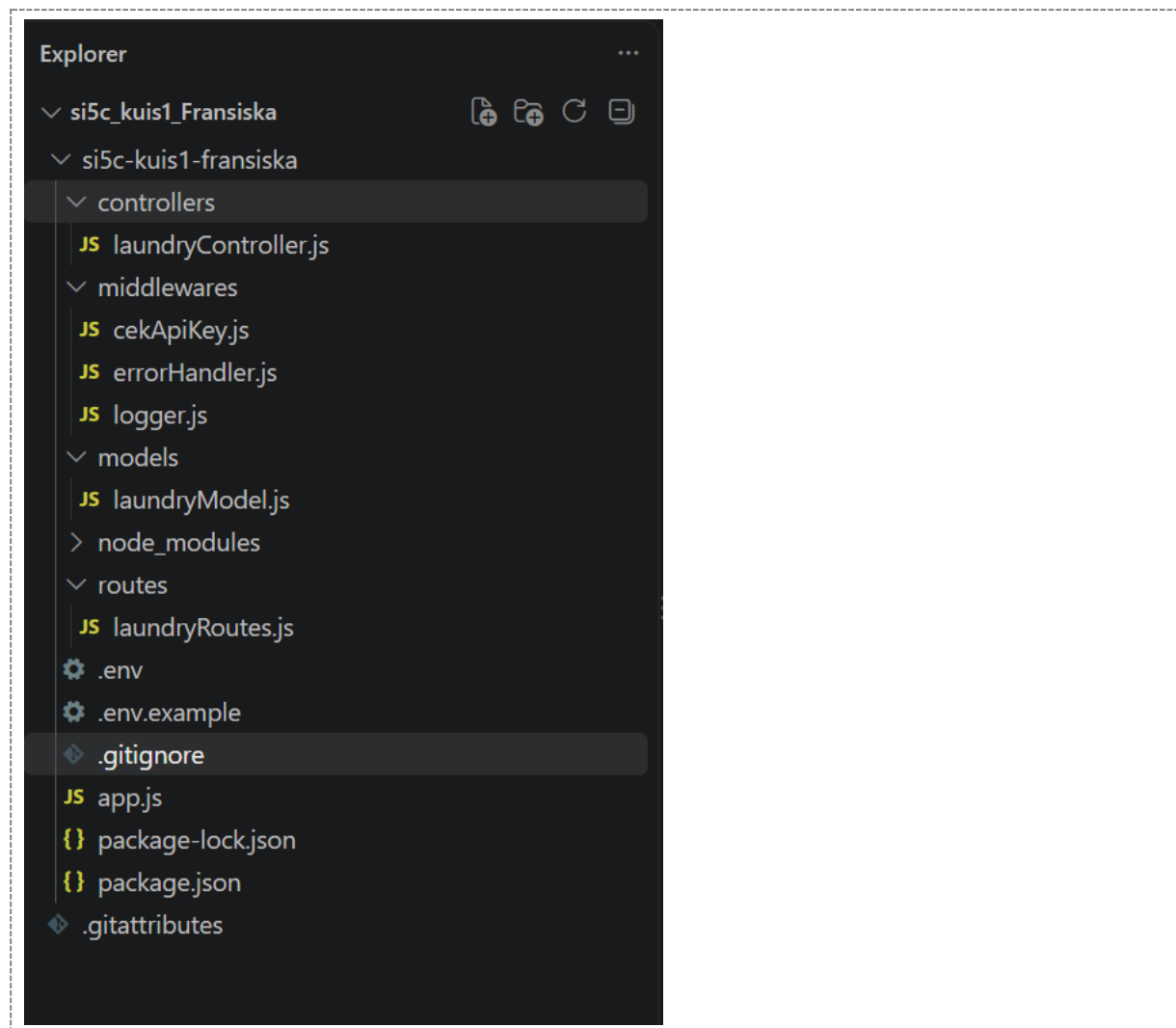
Express digunakan untuk membuat server dan routing RESTful API. CORS digunakan agar API dapat menerima request dari origin lain. Dotenv digunakan untuk membaca konfigurasi seperti PORT dan API_KEY dari file .env. Node.js digunakan sebagai runtime JavaScript; versi yang digunakan adalah Node.js 24.21.0.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder routes berisi pemetaan URL ke controller. Folder controllers menangani request, validasi, dan response. Folder models mengelola data laundry dan proses pengolahannya. Folder middlewares berisi logger, pengecekan API key, dan error handler terpusat.

2.2 Ringkasan tugas setiap berkas

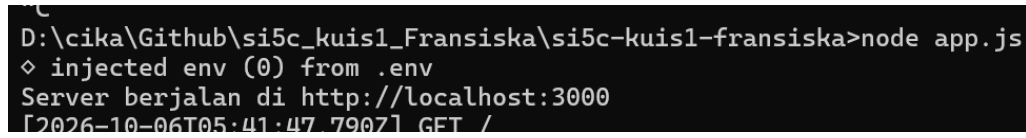
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/laundryModel.js	Menyimpan array data laundry dan fungsi getAll, getByld, create, update, dan remove tanpa menggunakan req/res.
controllers/laundryController.js	Mengambil data dari req, melakukan validasi, memanggil model, lalu mengirim response JSON dengan status yang sesuai.
routes/laundryRoutes.js	Menggunakan express.Router() untuk memetakan GET, POST, PUT, dan DELETE ke fungsi controller; POST/PUT/DELETE dilindungi cekApiKey.
middlewares/logger.js	Mencatat waktu, method, dan URL setiap request sebelum request diteruskan dengan next().
middlewares/cekApiKey.js	Memeriksa header x-api-key dan membandingkannya dengan API_KEY pada .env.
middlewares/errorHandler.js	Menangani route yang tidak ditemukan dan mengubah JSON rusak menjadi response 400 JSON.
app.js	Memuat dotenv, express, cors, middleware, route /laundry, error handler, dan menjalankan server pada PORT.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah npm start atau npm run dev dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
D:\cika\Github\si5c_kuis1_Fransiska\si5c-kuis1-fransiska>node app.js
◇ injected env (0) from .env
Server berjalan di http://localhost:3000
[2026-10-06T05:41:47.790Z] GET /
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port berasal dari variabel PORT pada .env, yaitu 3000. Jika PORT tidak tersedia, app.js menggunakan nilai cadangan 3000. Jika port sudah dipakai aplikasi lain, server akan gagal listen dan port harus diganti atau aplikasi yang memakai port tersebut dihentikan.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

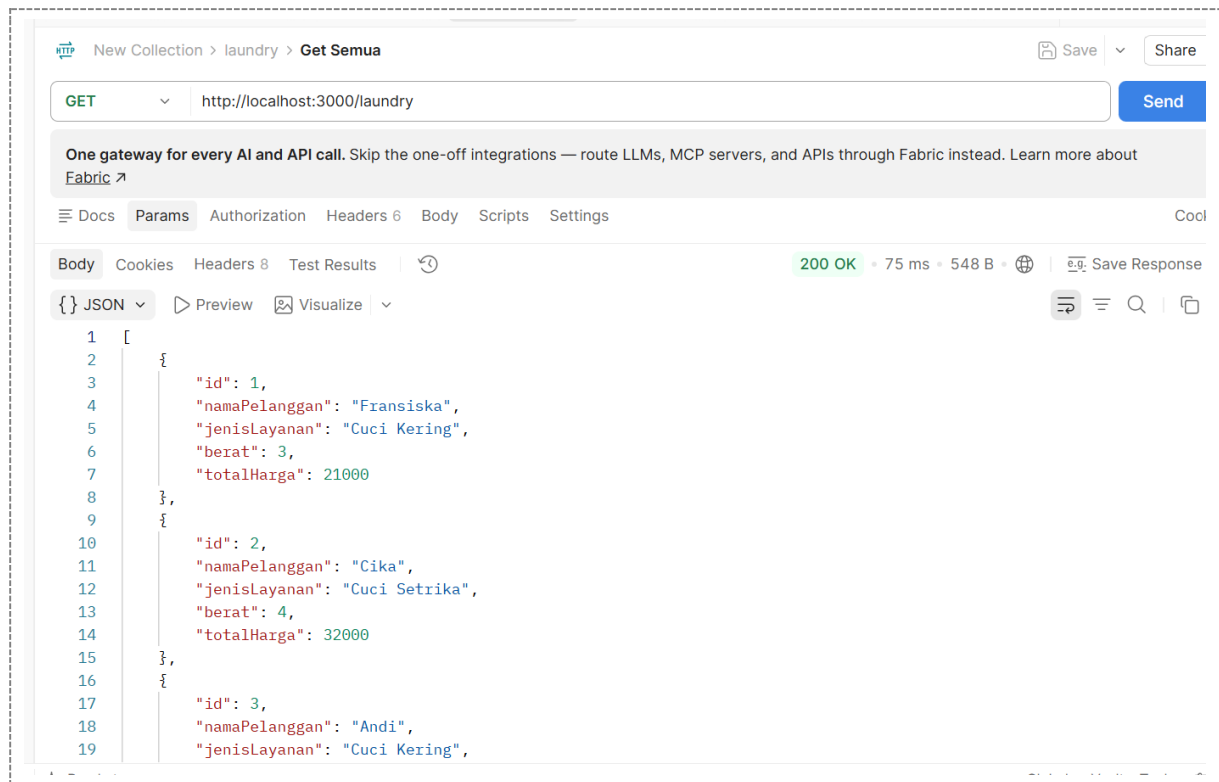
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /laundry
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data laundry.
Status hasil uji	200 OK
Lulus?	Ya



Gambar 4. Hasil GET semua data

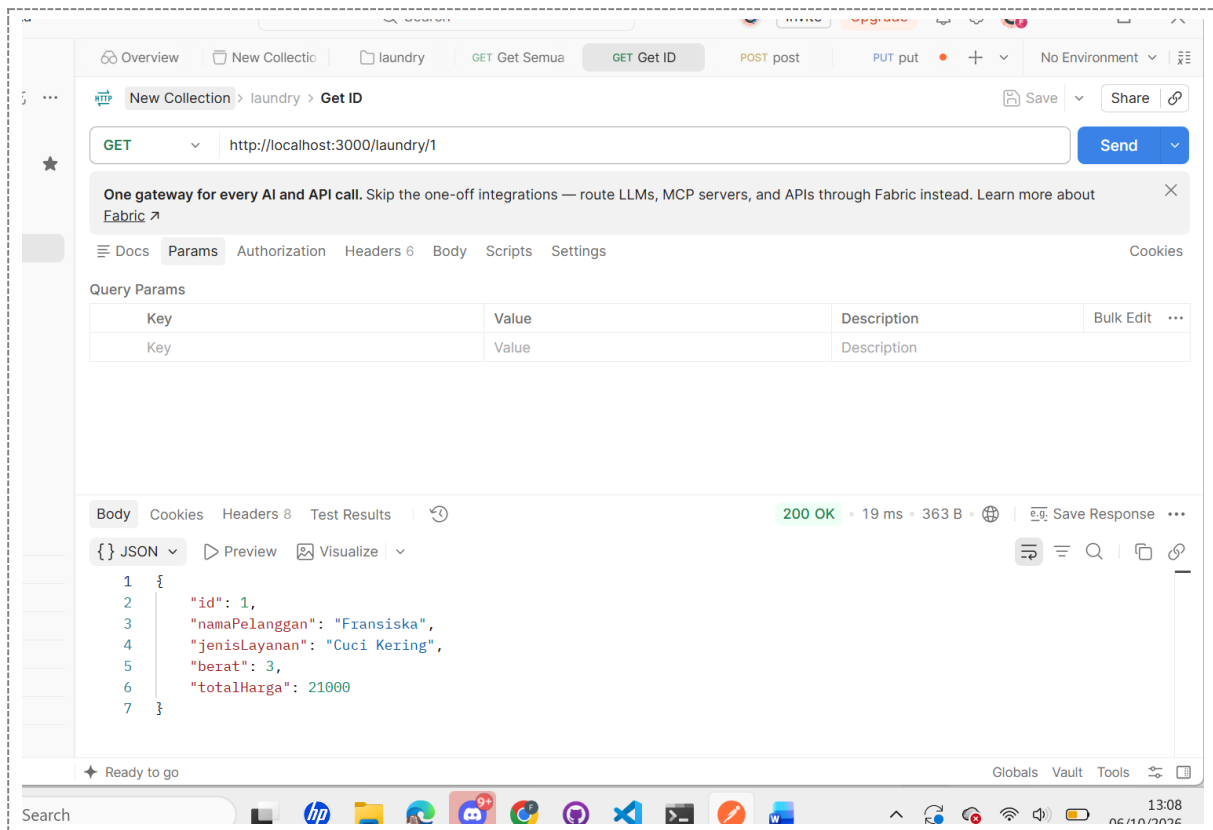
Penjelasan:

Status 200 menunjukkan request berhasil. Route GET /laundry memanggil laundryController.getAll(), lalu controller memanggil laundryModel.getAll() dan mengirim seluruh data sebagai JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /laundry/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 1.
Status hasil uji	200 OK
Lulus?	Ya



Gambar 5. Hasil GET berdasarkan ID

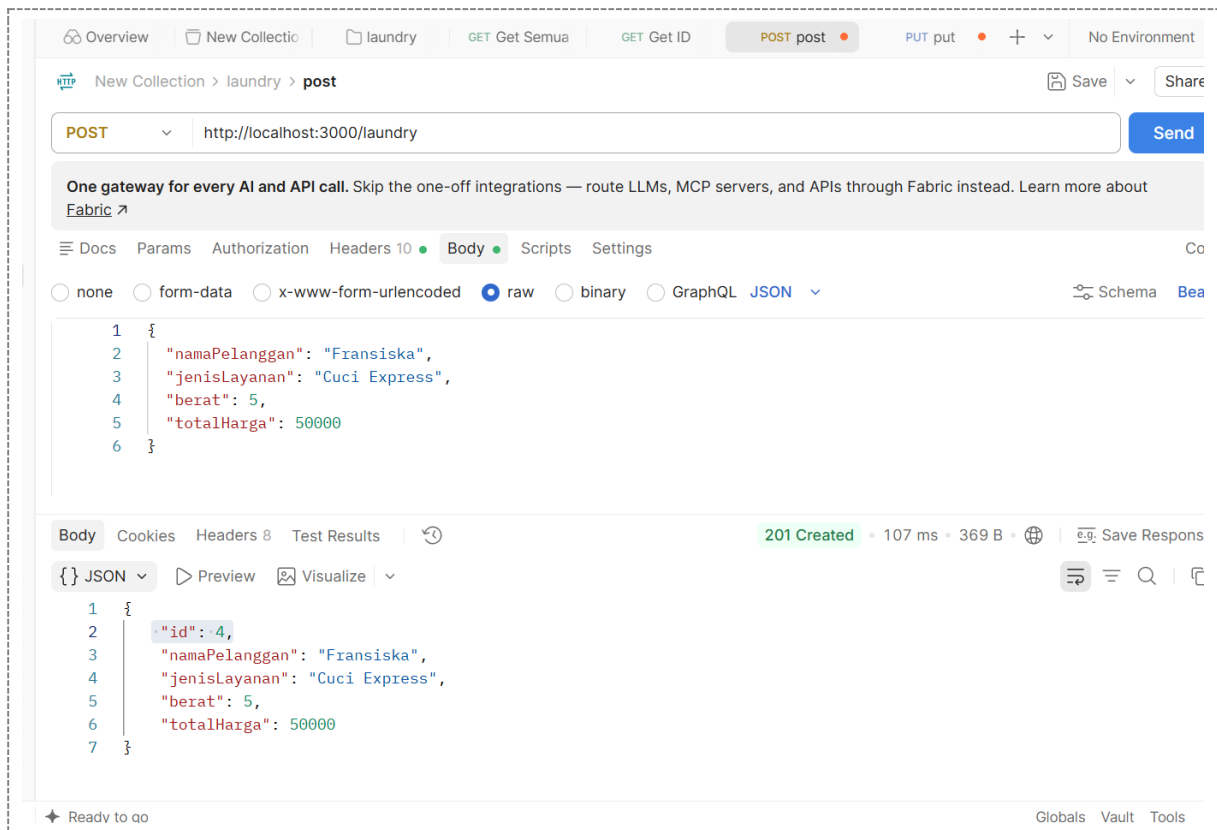
Penjelasan:

ID diambil dari req.params.id pada controller. Nilainya dikonversi menjadi angka saat model menjalankan Number(id), karena ID pada data laundry berupa number.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (<i>request</i>)	POST /laundry
Header	x-api-key: fransiska123 Content-Type: application/json
Body	{ "namaPelanggan": "Fransiska", "jenisLayanan": "Cuci Express", "berat": 5, "totalHarga": 50000 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created
Lulus?	Ya



Gambar 6. Hasil POST data baru

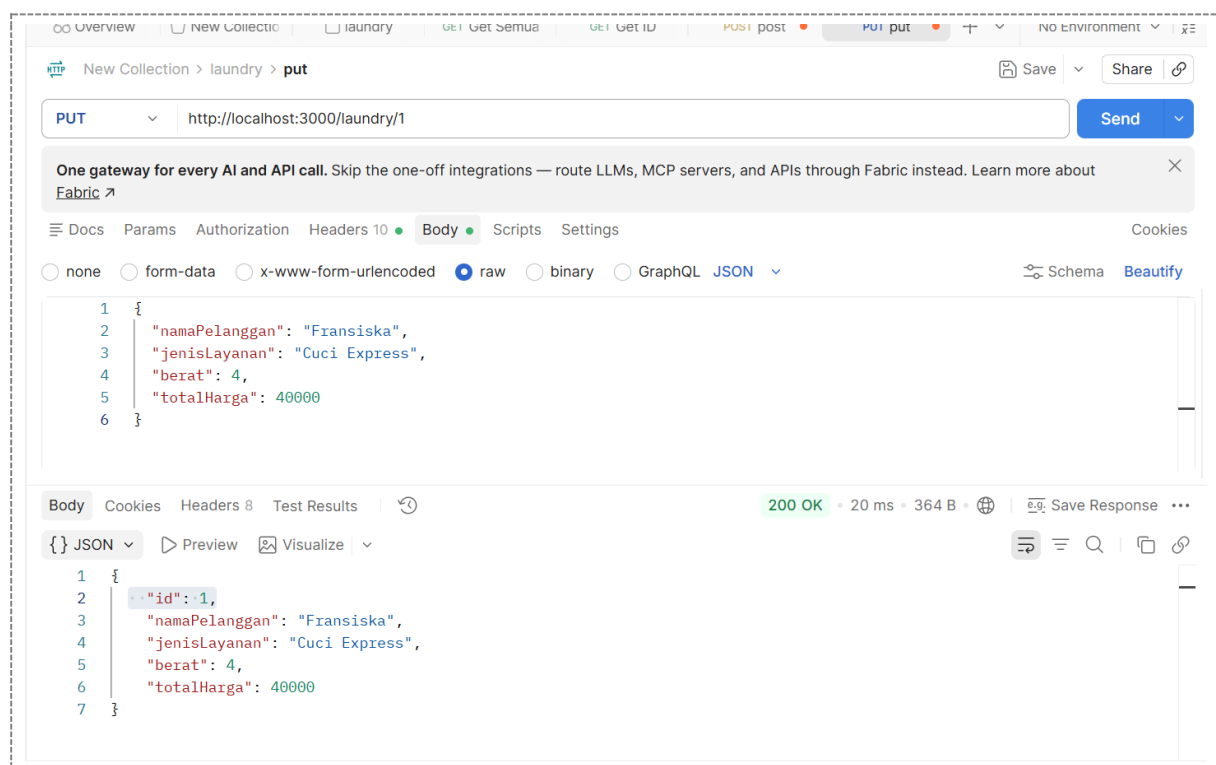
Penjelasan:

Header `x-api-key` diperiksa oleh middleware `cekApiKey` sebelum controller dijalankan. `express.json()` membaca body JSON menjadi `req.body`. Status 201 digunakan karena request berhasil membuat data baru.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /laundry/4
Header	x-api-key: fransiska123 Content-Type: application/json
Body	{ "namaPelanggan": "Fransiska", "jenisLayanan": "Cuci Express", "berat": 4, "totalHarga": 40000 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK
Lulus?	Ya



Gambar 7. Hasil PUT mengubah data

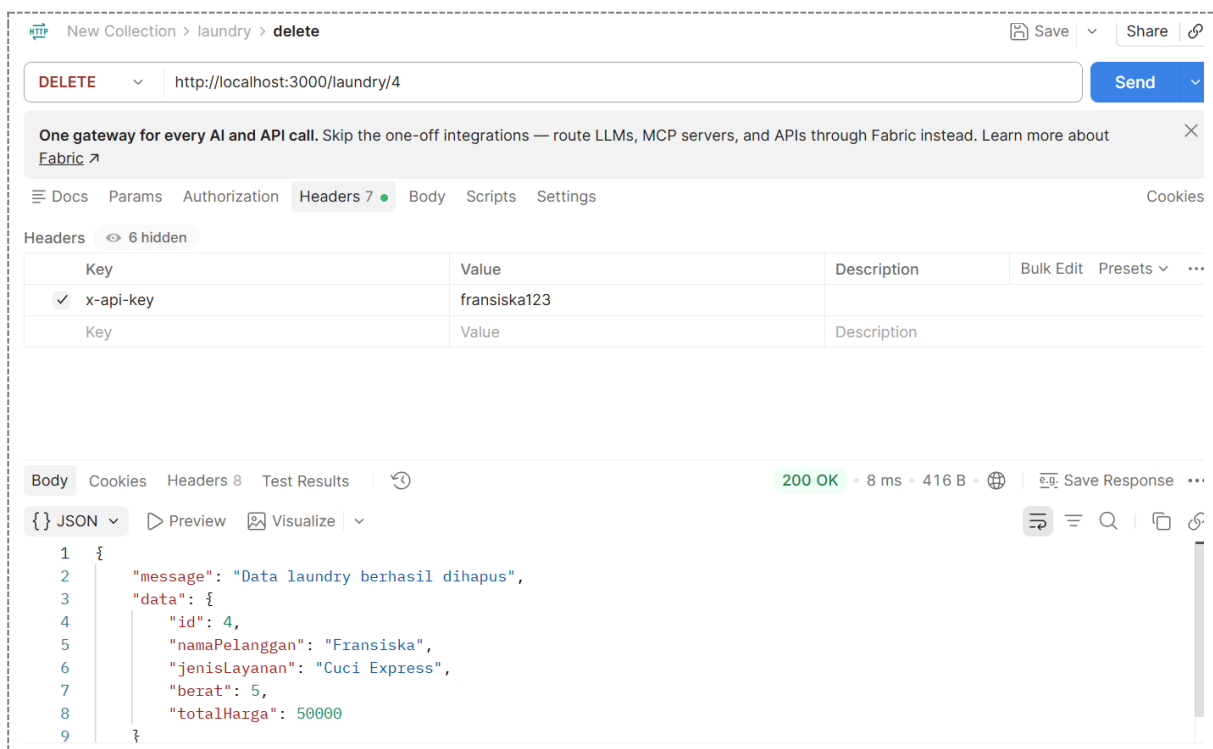
Penjelasan:

Data dengan ID 4 yang dibuat pada T03 diubah pada field jenisLayanan, berat, dan totalHarga. Perubahan data dilakukan pada layer Model melalui fungsi update(), sedangkan Controller menangani request dan response.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /laundry/4
Header	x-api-key: fransiska123
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	Ya



Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 berarti data berhasil dihapus tetapi server tidak mengirim isi response. Karena itu controller menggunakan `res.status(204).send()` tanpa body.

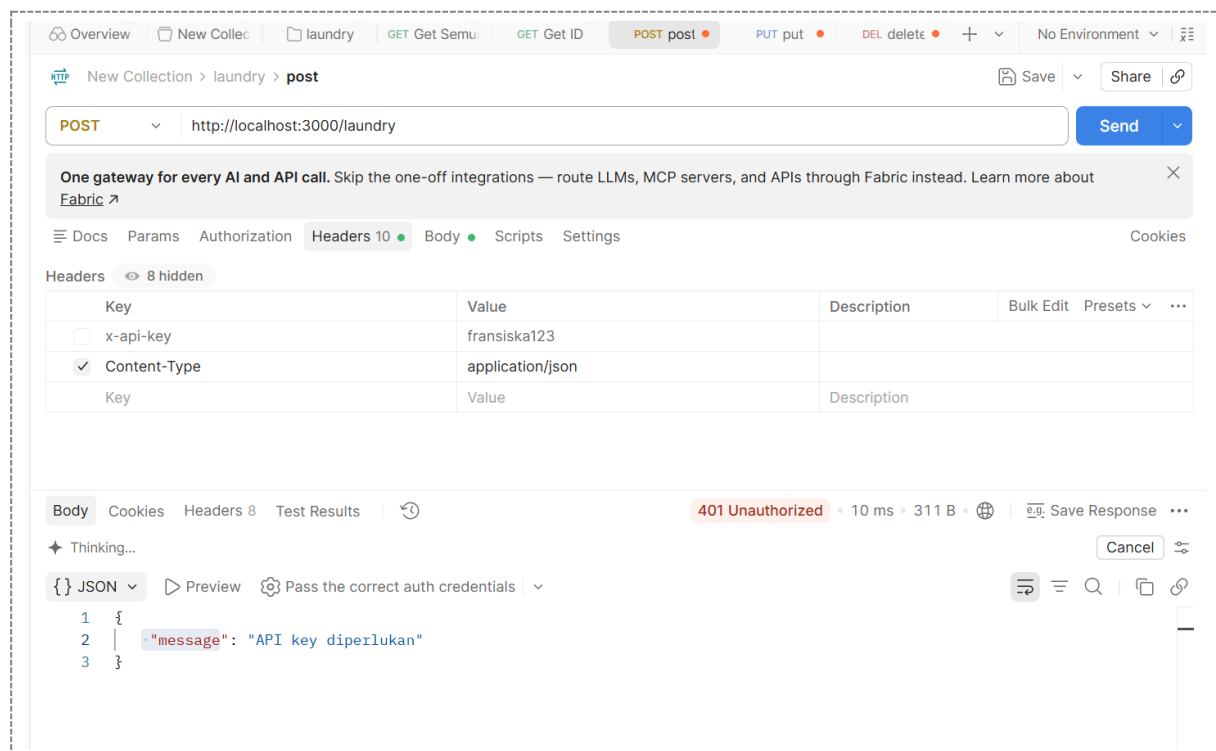
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /laundry (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "namaPelanggan": "Cika", "jenisLayanan": "Cuci Kering", "berat": 2, "totalHarga": 14000 }</pre>
Hasil yang diharapkan	401 Unauthorized; API key diperlukan.
Status hasil uji	401 Unauthorized
Lulus?	Ya



Gambar 9. POST tanpa API key ditolak

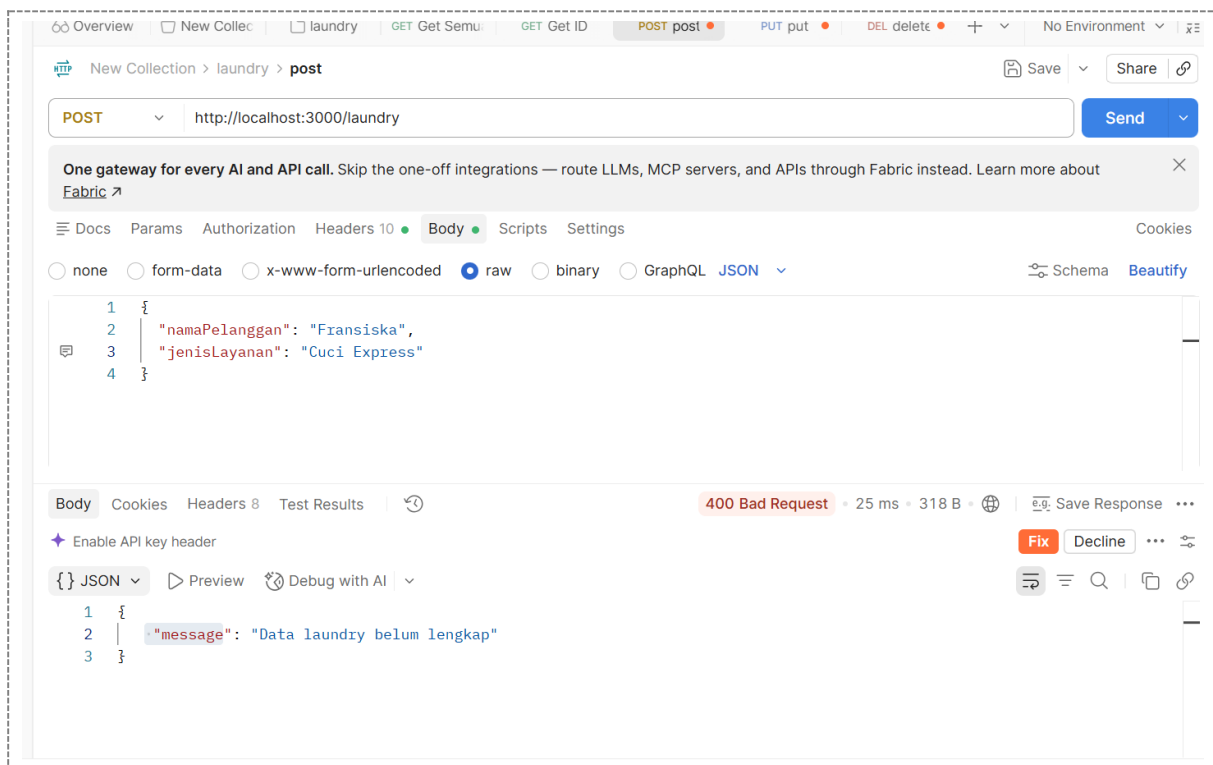
Penjelasan:

Middleware cekApiKey menolak request ketika header x-api-key tidak ada. Karena middleware mengembalikan response 401 dan tidak memanggil next(), controller create() tidak sempat dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /laundry
Header	x-api-key: fransiska123 Content-Type: application/json
Body	{ "namaPelanggan": "Cika" }
Hasil yang diharapkan	400 Bad Request; data laundry belum lengkap.
Status hasil uji	400 Bad Request
Lulus?	Ya



Gambar 10. POST dengan body tidak lengkap

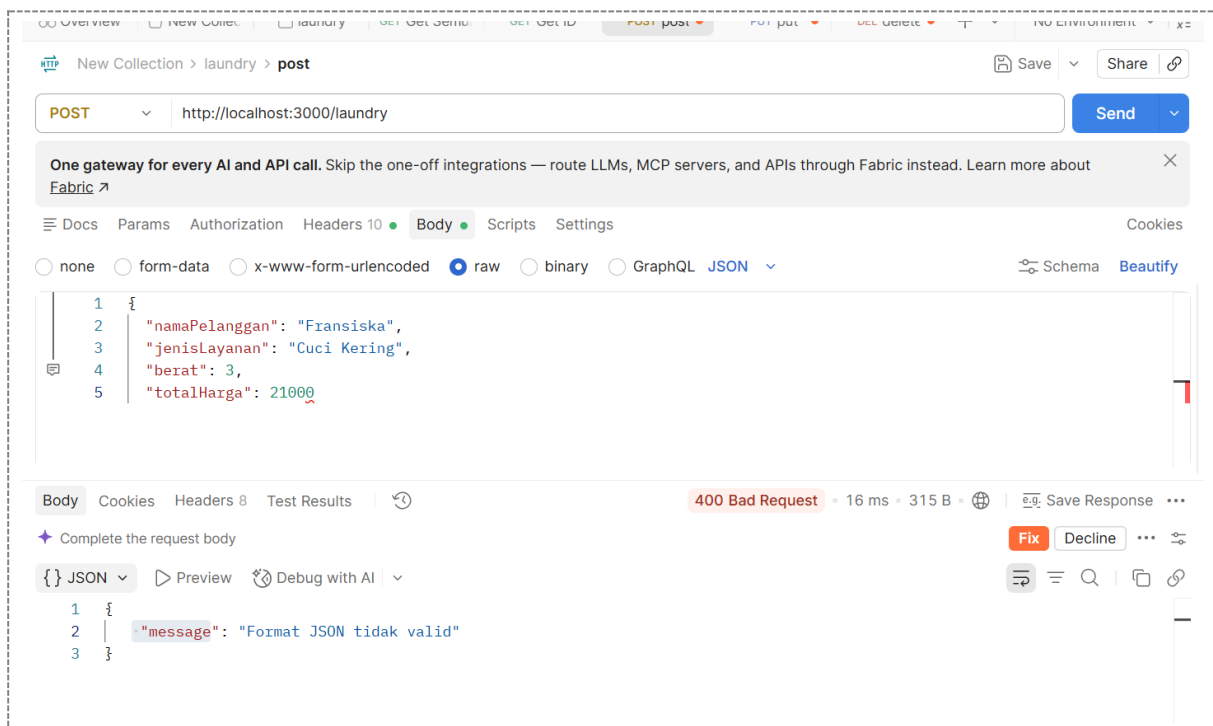
Penjelasan:

Validasi data dilakukan pada Controller. Controller memeriksa namaPelanggan, jenisLayanan, berat, dan totalHarga. Jika ada yang belum diisi, controller mengembalikan 400 Bad Request karena request dari client tidak memenuhi data wajib.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /laundry
Header	x-api-key: fransiska123 Content-Type: application/json
Body	{ "namaPelanggan": "Cika", "jenisLayanan": "Cuci Kering", "berat": 2, "totalHarga": 14000, }
Hasil yang diharapkan	400 Bad Request; "Format JSON tidak valid".
Status hasil uji	400 Bad Request
Lulus?	Ya



Gambar 11. POST dengan JSON rusak

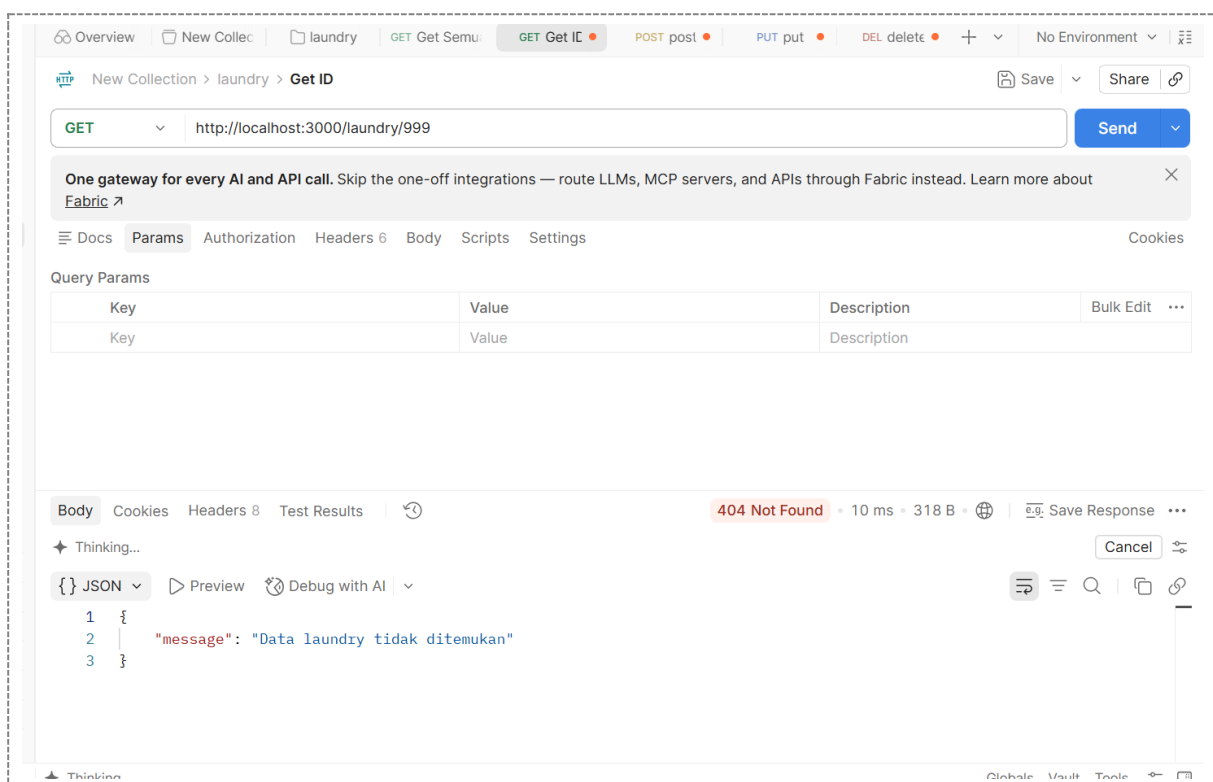
Penjelasan:

JSON yang rusak dideteksi oleh `express.json()` sebelum controller menerima request. Error kemudian diteruskan ke errorHandler, yang mengenali `entity.parse.failed` dan mengubahnya menjadi response JSON 400 dengan pesan Format JSON tidak valid.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (<i>request</i>)	GET /laundry/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; data laundry tidak ditemukan.
Status hasil uji	404 Not Found
Lulus?	Ya



Gambar 12. GET ID tidak ada

Penjelasan:

Route GET /laundry/:id memanggil getById() pada controller. Controller meminta model mencari ID 99. Karena model mengembalikan null/undefined, controller mengirim response 404 dengan pesan Data laundry tidak ditemukan.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	Ya
T05	Menghapus data	204	204 No Content	Ya
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	[Link GitHub repository si5c_kuis1_Fransiska]
Pesan commit terakhir	refactor API laundry ke arsitektur backend terstruktur
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route bertugas menentukan alamat dan method request, misalnya `router.get('/', laundryController.getAll)`. Controller menangani request, validasi, dan response, misalnya fungsi `create` pada `laundryController`. Model mengelola data, misalnya fungsi `laundryModel.create()` yang menambahkan data laundry baru tanpa menggunakan `req` atau `res`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan error lebih rapi karena kesalahan tertentu dapat diproses di satu tempat. Pada pengujian JSON rusak, `express.json()` menghasilkan error lalu `errorHandler` mengubahnya menjadi response 400 JSON dengan pesan Format JSON tidak valid. Untuk ID yang tidak ditemukan, controller mengirim 404 sesuai kondisi data.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100